

DOCUMENTATION DE RÉFÉRENCE

Humanitarian Data Platform

Version finale 3.0.0

Spécifications · installation · architecture · API · sécurité · exploitation ·
livrables

Édition du 15 août 2026

Sommaire

- 01 Présentation générale
- 02 Cahier des charges
- 03 Guide utilisateur
- 04 Installation
- 05 Architecture
- 06 Référence API
- 07 Passerelle GitHub
- 08 Migration depuis 2.5.0
- 09 Sécurité
- 10 Développement
- 11 Dépannage
- 12 Livrables
- 13 Historique

Présentation générale

Humanitarian Data Platform 3.0.0 — version finale

Application locale Windows pour rechercher des données humanitaires publiques, télécharger leurs ressources, les organiser par projets et automatiser les acquisitions géographiques.

Fonctionnalités finales 3.0.0

- **Exécution Python/R bornée** : versions immuables, rapports SHA-256, délai et sortie limités, runners sans privilège et sans réseau ; R reste facultatif.
- **Veille RSS officielle** : registre borné aux quatre flux ReliefWeb vérifiés, abonnements par projet, lecture immédiate ou planifiée, déduplication et parsing XML durci.
- **Chronologie Gantt** : acquisitions, planifications et exécutions de scripts sont réunies dans une vue temporelle du projet.
- **Cartographie locale** : import GeoJSON dans PostGIS, rendu Leaflet local et exports prêts pour QGIS et R. Les tuiles OpenStreetMap ne sont chargées qu'après action explicite.
- **GitHub à droits minimaux** : politique de jeton affichée sans exposer le secret et recommandation d'un jeton finement granulé avec permission d'administration du dépôt uniquement lorsque la création est utilisée.
- **Passerelle REST GitHub locale** : lecture des dépôts, branches, commits, issues, pull requests, releases, workflows, contenus et quotas ; création d'issues et déclenchement de workflows verrouillés par défaut.

Leaflet 1.9.4 est embarqué : ouvrir l'interface ne provoque aucun chargement de code depuis un CDN.

État de validation final

- 68 tests Python réussis ;
- API principale : 47 chemins et 63 opérations OpenAPI ;
- passerelle GitHub : 11 chemins et 12 opérations OpenAPI ;
- JavaScript inline et Compose/YAML analysés avec succès ;
- runner C17 compilé strictement et essayé sur une exécution isolée ;
- 39 fichiers du payload reconstruits octet pour octet ;
- installateur PE32+ GUI x64 avec ASLR, NX et haute entropie.

La recette Windows 10/11 avec Docker Desktop reste à exécuter sur une machine Windows réelle ; elle n'est pas confondue avec les validations locales ci-dessus.

Paramétrage des sources et bibliothèque

- **Paramètres API versionnés** : contrats distincts pour les 7 connecteurs, validés côté serveur et rendus dynamiquement dans l'interface.

- **Deux niveaux de configuration** : activation/délai/reprises globalement, paramètres, limites et planification par projet et par source.
- **Prévisualisation sûre** : URL et commande assainies affichées avant appel ; aucun secret n'est retourné.
- **Provenance enrichie** : paramètres effectifs archivés avec les acquisitions et planifications ; bibliothèque filtrable par source, format, sujet, organisme, localisation et dates.
- **Mise à niveau** : migrations idempotentes enregistrées et conservation des variables inconnues de `.env`, des fichiers et du volume PostgreSQL.
- **Windows** : EXE 3.0.0 réellement compilé en PE32+ GUI x64 ; détection de mise à niveau, sauvegarde de `.env`, ASLR/NX et payload contrôlés. L'essai sur Windows avec Docker reste obligatoire.

Socle conservé de la version 2.5.0

- **Sources sanitaires mondiales** : catalogue intégré de 18 sources avec organisme, couverture, domaines, accès et liens officiels.
- **Cinq nouveaux connecteurs** : WHO Global Health Observatory, World Bank Health Indicators, UNICEF Data Warehouse (SDMX), UN Global SDG Indicators et DHS Program Indicator Data rejoignent HDX et ReliefWeb.
- **Capacités explicites** : l'interface distingue 7 API interrogeables et planifiables de 11 portails de référence dont l'accès reste manuel, variable ou soumis à inscription.
- **Provenance inchangée** : la réponse distante originale est archivée avec son empreinte SHA-256 avant normalisation ; les téléchargements restent bornés par les préférences du projet.
- **Liste des téléchargements officiels** : COD-AB (limites administratives) et COD-PS (statistiques de population infranationales) sont sélectionnables par projet. COD-CS est visible mais désactivé tant que son registre vérifié est vide ; COD-HP est signalé comme retiré par OCHA.
- **Liste géographique contrôlée** : chaque option est un pays ou une zone à la fois dans ONU M49 et dans les groupes HDX canoniques de toutes les familles sélectionnées. Le 7 août 2026, HDP vérifie 163 options COD-AB, 146 COD-PS et 143 dans leur intersection.
- **Synchronisation atomique** : HDP ne télécharge pas seulement une partie des familles demandées si le catalogue change. La réponse CKAN et la décision restent archivées pour diagnostic.
- **Formats adaptés** : format géospatial choisi pour COD-AB ; ressources CSV/XLSX pour COD-PS. La famille est conservée avec la provenance locale.
- **Dépôt GitHub par projet** : paramètres distincts (propriétaire, nom, description et visibilité) et création réelle après confirmation. Le jeton reste global dans `.env` et n'est jamais exposé par l'API.
- **Migration explicite** : un ancien profil déjà limité à un pays est conservé ; les anciens profils monde/région sont suspendus jusqu'au choix dans la nouvelle liste.

- **Reprise progressive** : les ressources déjà téléchargées ne consomment plus le quota du passage ; les suivantes sont reportées proprement.
- **Socle 2.0 conservé** : projets, ressources locales, préférences, scripts non exécutables et planifications restent isolés par projet.
- **Projets** : chaque projet possède ses acquisitions, ressources, préférences, scripts et planifications.
- **Téléchargement automatique** : option par recherche ou planification, avec limites de taille, de quantité et de formats.
- **Planificateur persistant** : exécutions périodiques à partir de 15 minutes et historique conservé dans PostgreSQL.
- **Données locales** : inventaire, téléchargement depuis l'interface, contrôle SHA-256 et suppression du fichier avec conservation de la trace.
- **Scripts par projet** : création et modification de contenu Python, R, SQL ou autre. Seuls Python et R sont exécutables ; les autres langages restent stockés.
- **Migration 1.5** : les acquisitions existantes rejoignent automatiquement le « Projet par défaut » ; .env, le volume PostgreSQL et data/ sont conservés.

Installation Windows

- Téléchargez HumanitarianDataPlatform_Windows_v3.0.0.zip.
- Décompressez l'archive.
- Vérifiez l'empreinte :

```
``powershell Get-FileHash .\HumanitarianDataPlatform_Setup_Native_GUI_v3.0.0.exe -Algorithm
SHA256 ``
```

- Comparez-la au fichier .exe.sha256, puis lancez l'installateur.
- Docker Desktop est nécessaire ; le module R reste facultatif.

L'application s'ouvre sur `http://localhost:8080` ou sur un port libre entre 18080 et 18279. Elle reste liée à 127.0.0.1.

Si les paramètres d'un projet affichent « GITHUB_TOKEN absent : la création reste indisponible. », utilisez le [correctif Windows automatisé](#). Il configure le secret par saisie masquée, recrée uniquement l'API et vérifie le résultat sans révéler le jeton.

Documentation

- [Guide utilisateur](#)
- [Installation et mise à niveau](#)
- [Architecture, données et API](#)
- [Migration vers 3.0.0](#)
- [Sécurité et confidentialité](#)
- [Passerelle REST GitHub](#)
- [Cahier des charges final](#)
- [Référence API](#)
- [Prompt global de production](#)
- [Développement et validation](#)
- [Dépannage](#)
- [Inventaire des livrables et empreintes](#)
- [Journal des versions](#)

- [Prompt historique de reconstruction de l'état v2.5.0](#)

FastAPI publie aussi une documentation interactive locale sur /docs lorsque l'application tourne.

Le journal texte CHANGELOG_HDP.log est également embarqué dans l'installateur et copié à la racine de l'application Windows.

Sources prises en charge

Source	Recherche / planification	Accès
HDX / CKAN	Oui	API publique ; ressources et COD selon la licence du jeu
ReliefWeb	Oui	RELIEFWEB_APPNAME pré-approuvé requis
WHO Global Health Observatory	Oui	API OData publique
World Bank Health Indicators	Oui	Indicators API v2 publique
UNICEF Data Warehouse	Oui	API SDMX publique
UN Global SDG Indicators	Oui	API publique
DHS Program Indicator Data	Oui	Indicateurs agrégés publics ; microdonnées sur inscription

L'onglet « Sources sanitaires » référence aussi WHO Mortality Database, WHO GLASS, WHO FluNet/FluID, WHO Global Health Estimates, UNAIDS AIDSinfo, IHME GHDx, UNICEF MICS, UN World Population Prospects, Global.health, WorldPop et Our World in Data. Ces portails ne sont pas présentés comme des API actives.

Voir les documentations officielles de l'[Action API CKAN](#) et de l'[API ReliefWeb V2](#). Les quotas, licences et conditions d'utilisation de chaque source restent applicables.

Organisation du dépôt

```
source/ code FastAPI, interface, installateur Win32 et tests
docs/ documentation Markdown consultable dans GitHub
dist/v3.0.0/ livrables finaux, installateur, documentation et empreintes
dist/v2.5.0/ installateur, archives, empreintes et prompt de reprise
dist/v2.4.0/ livrables historiques intacts
dist/v2.3.2/ livrables historiques intacts
dist/v2.3.1/ livrables historiques intacts
dist/v2.0/ livrables historiques intacts
dist/v1.5/ livrables historiques intacts
tools/ génération de la notice PDF
```

Sécurité et licence

HDP 3.0.0 est une application locale mono-utilisateur, sans authentification, et ne doit pas être exposée directement sur Internet. Les téléchargements refusent les URL non HTTP(S), les identifiants intégrés et les destinations réseau non publiques ; chaque projet impose des limites. Les scripts exécutés doivent être considérés comme du code local de confiance, même si les runners sont sans réseau et bornés. Les groupements M49 sont statistiques et n'impliquent aucune position politique. Un dépôt GitHub public ne doit être créé qu'après vérification de son contenu et de sa licence.

Aucune licence HDP explicite n'est incluse. Le dépôt doit rester privé tant qu'une licence n'a pas été choisie et ajoutée.

Cahier des charges

Cahier des charges final - Humanitarian Data Platform 3.0.0

1. Objet

Humanitarian Data Platform, ou HDP, est une application Windows locale destinée à acquérir, organiser, vérifier, planifier, cartographier et traiter des données humanitaires, épidémiologiques et sanitaires mondiales. L'installation est effectuée par un exécutable graphique natif ; l'utilisation quotidienne se fait dans le navigateur.

2. Périmètre garanti

- utilisateur unique sur un poste Windows 10/11 x64 ;
- interface et services liés à 127.0.0.1 ;
- Docker Desktop et WSL 2 comme socle d'exécution ;
- FastAPI/Python pour l'API principale ;
- PostgreSQL 16/PostGIS 3.4 pour les métadonnées et géométries ;
- R/plumber facultatif ;
- scripts Python et R uniquement, exécutés hors ligne ;
- compatibilité de mise à niveau ciblée depuis HDP 2.5.0.

Sont exclus : exposition Internet ou LAN, multi-utilisateur, authentification centralisée, TLS public, exécution shell/SQL, traitement de code hostile, stockage de secrets dans la base ou publication GitHub automatique de données.

3. Sources

Connecteurs interrogeables et planifiables

- HDX/CKAN ;
- ReliefWeb API v2 ;
- WHO Global Health Observatory ;
- World Bank Health Indicators ;
- UNICEF Data Warehouse/SDMX ;
- UN Global SDG Indicators ;
- DHS Program Indicator Data.

Chaque connecteur expose un contrat de paramètres versionné. HDP distingue les réglages globaux des réglages propres à chaque projet. Les réponses distantes brutes sont archivées avant normalisation, avec empreinte SHA-256.

Références sanitaires

Le catalogue intégré distingue les API actives des portails sans API publique stable ou dont l'accès varie selon les données : WHO Mortality, GLASS, FluNet/FluID, Global Health Estimates, UNAIDS, IHME/GHDx, MICS, World Population Prospects, Global.health, WorldPop et Our World in Data.

Flux RSS

Le registre RSS est borné aux quatre flux officiels ReliefWeb : mises à jour, catastrophes, emplois et formations. Les réponses sont limitées à 2 Mio, analysées avec defusedxml, sans DTD/entité, et dédoublées en base.

4. Fonctions attendues

Projets

- créer, modifier et archiver logiquement un projet ;
- isoler préférences, acquisitions, ressources, scripts et planifications ;
- conserver un « Projet par défaut » stable pour la migration.

Acquisitions et ressources

- rechercher les sept connecteurs ;
- prévisualiser les requêtes sans appel réseau et sans secret ;
- archiver le JSON brut et sa provenance ;
- télécharger facultativement les ressources ;
- appliquer taille, quantité et formats autorisés ;
- refuser URL privées/locales, identifiants intégrés et redirections non sûres ;
- écrire par fichier temporaire puis renommage atomique ;
- vérifier et exposer l'empreinte SHA-256 ;
- supprimer un fichier avec confirmation tout en conservant sa trace.

Géodonnées

- proposer COD-AB et COD-PS officiels ;
- afficher COD-CS comme indisponible tant que le registre vérifié est vide ;
- signaler COD-HP comme retiré ;
- limiter la zone à l'intersection ONU M49 et groupes HDX canoniques ;
- effectuer une synchronisation atomique des familles sélectionnées ;
- importer un GeoJSON borné dans PostGIS SRID 4326 ;
- afficher les couches avec Leaflet 1.9.4 embarqué ;
- ne charger les tuiles OpenStreetMap qu'après action explicite ;
- exporter GeoJSON et scripts QGIS/R.

Scripts

- créer une version immuable à chaque modification ;
- autoriser uniquement Python et R ;
- utiliser un runner par langage, sans shell et sans réseau ;
- limiter durée, sortie, processus, fichiers, mémoire et CPU ;
- conserver statut, sortie, erreurs, horodatages et rapport JSON SHA-256 ;
- considérer tous les scripts comme code local de confiance.

Planification et chronologie

- planifier acquisitions, synchronisations géographiques et RSS ;
- intervalle minimal de 15 minutes ;

- revendiquer les travaux avec verrou transactionnel ;
- conserver l'historique ;
- afficher acquisitions, passages, scripts et échéances dans un Gantt.

GitHub

- stocker le jeton uniquement dans `.env` ;
- créer un dépôt après confirmation explicite, privé par défaut ;
- recommander un jeton finement granulé à droits minimaux ;
- fournir une passerelle REST locale pour dépôt, branches, commits, issues, pull requests, releases, workflows, contenus et quotas ;
- désactiver par défaut la création d'issues et le dispatch de workflows.

5. Architecture Docker

Service	Rôle	Exposition
db	PostgreSQL/PostGIS	interne
api	FastAPI, interface, planificateur	127.0.0.1:8080 ou port choisi
runner-python	exécution Python hors ligne	aucune, <code>network_mode: none</code>
github-api	passerelle REST GitHub	127.0.0.1:8091
r-service	résumés R/plumber facultatifs	interne, profil analytics
runner-r	exécution R hors ligne	aucune, profil analytics

Le nom Compose et le volume PostgreSQL historique doivent rester stables.

6. Installation et mise à niveau

L'installateur Win32 doit :

- rester graphique et réactif pendant les tâches longues ;
- détecter winget, Docker, Git et VS Code ;
- proposer les tiers sans sélection automatique ;
- choisir 8080 ou un port libre entre 18080 et 18279 ;
- préserver `.env`, ses variables inconnues, `data/` et le volume PostgreSQL ;
- créer `.env.backup-before-v3.0.0` avant réécriture ;
- construire les services requis et ouvrir le navigateur après santé positive ;
- créer les raccourcis Windows prévus ;
- ne jamais lancer de purge Docker ou `down -v`.

7. Sécurité et conformité

- aucun secret dans les réponses, journaux, archives ou dépôt ;
- validation des entrées et bornes explicites ;
- requêtes distantes limitées aux hôtes/URL prévus ;
- conteneurs d'exécution non privilégiés, capacités supprimées et lecture seule ;
- tuiles OSM opt-in avec attribution ;
- données humanitaires sensibles exclues sans évaluation dédiée ;
- dépôt privé tant qu'aucune licence HDP explicite n'est adoptée.

8. Critères d'acceptation

- suite Python intégralement réussie ;
- compilation de tous les modules Python ;
- JavaScript analysable ;
- YAML Compose valide ;
- contrats OpenAPI générables ;
- runner C compilé en C17 strict et testé sur succès/dépassement de délai ;
- payload reconstruit octet pour octet ;
- installateur PE32+ GUI x64 avec ASLR/NX ;
- archives ZIP intègres et sommes SHA-256 vérifiées ;
- aucune opération destructive ou secret versionné ;
- commit final descendant de main, publication non forcée et vérification du commit distant.

La recette finale Windows/Docker doit être consignée séparément lorsqu'elle ne peut pas être exécutée dans l'environnement de construction.

Guide utilisateur

Guide utilisateur - HDP 3.0.0 final

Projet actif

Le sélecteur en haut de l'interface détermine le projet actif. Toutes les recherches, ressources, préférences, scripts et planifications affichés appartiennent à ce projet.

Créez un projet dans **Projets & préférences**, puis décrivez son objectif. Le « Projet par défaut » accueille les acquisitions anciennes et ne peut pas être archivé.

Recherche et téléchargement automatique

Dans **Recherche** :

- choisissez HDX/CKAN, ReliefWeb, WHO/GHO, World Bank Health Indicators, UNICEF/SDMX, UN/SDG ou DHS ;
- saisissez les mots-clés et le nombre maximal de résultats ;
- activez, si besoin, le téléchargement automatique ;
- lancez l'acquisition.

La réponse JSON est toujours archivée avec son empreinte. Si le téléchargement est actif, les ressources présentes dans les métadonnées sont traitées dans l'ordre, jusqu'à la limite du projet. Un fichier trop grand, un format exclu ou une destination réseau privée est ignoré ou signalé en erreur sans annuler la provenance de l'acquisition.

Sources sanitaires

L'onglet **Sources sanitaires** inventorie 18 sources mondiales. La pastille **API active** signifie que la recherche et la planification sont intégrées à HDP. La pastille **Référence** ouvre un portail officiel sans prétendre qu'une API publique stable est disponible. La fiche indique aussi les domaines, les modalités d'accès et la nécessité éventuelle d'un compte.

Les cinq nouveaux connecteurs recherchent des indicateurs ou flux, pas des microdonnées individuelles. Pour DHS et MICS, une inscription distincte reste nécessaire lorsque l'utilisateur souhaite accéder aux microdonnées.

Paramètres des sources

La rubrique **Paramètres des sources** sépare deux périmètres :

- les réglages globaux du connecteur : activation, délai maximal, nouvelles tentatives et délai initial de reprise ;
- les réglages du projet actif : activation de la source, paramètres propres à son API, limite, téléchargement et valeurs par défaut de planification.

Choisissez une source : le formulaire est construit depuis son contrat versionné. **Prévisualiser la requête** valide les valeurs, enregistre le modèle du projet et affiche l'URL et la commande assainies sans appeler le service distant. Un secret éventuel est désigné par son nom de variable, jamais par sa valeur.

Bibliothèque locale

La rubrique **Données locales** filtre les ressources par source, format, sujet, organisme et localisation. Les nouvelles acquisitions conservent aussi la date de publication et les métadonnées normalisées disponibles. Les lignes plus anciennes restent visibles même si ces nouveaux champs sont vides.

Préférences

Chaque projet définit :

- activation du téléchargement automatique par défaut ;
- taille maximale de chaque fichier, de 1 Mio à 2 Gio ;
- nombre maximal de ressources par acquisition, de 1 à 100 ;
- formats autorisés, par exemple csv, json, geojson ; une liste vide accepte tous les formats.

Dépôt GitHub du projet

Dans **Projets & préférences**, renseignez le compte ou l'organisation, le nom du dépôt, sa description et sa visibilité. Enregistrez les paramètres, puis choisissez **Créer le dépôt**. Une confirmation explicite précède toujours l'appel GitHub.

Le champ compte peut rester vide pour utiliser le compte associé à GITHUB_TOKEN. Pour une organisation, le jeton doit autoriser la création de dépôts dans cette organisation. HDP initialise le dépôt avec un README GitHub, mais n'y publie ni fichiers locaux, ni ressources, ni scripts du projet.

Le jeton n'est pas un paramètre de projet : il est lu depuis .env, masqué dans l'installateur et absent des réponses API et des journaux HDP.

La passerelle REST GitHub complémentaire écoute localement sur `http://127.0.0.1:8091`. Ses fonctions de lecture sont disponibles lorsque le service est démarré. La création d'issues et le déclenchement de workflows restent désactivés tant que GITHUB_API_WRITE_ENABLED n'est pas explicitement défini à true. Voir [Passerelle REST GitHub](#).

Géodonnées ONU M49 et HDX

Le module présente quatre familles sous forme de liste :

- **COD-AB**, sélectionnable, pour les limites administratives ;
- **COD-PS**, sélectionnable, pour les statistiques de population infranationales ;
- **COD-CS**, visible mais désactivé tant que le registre vérifié embarqué est vide ;
- **COD-HP**, visible mais désactivé, car cette famille a été retirée par OCHA.

Choisissez ensuite :

- une ou plusieurs familles disponibles ;
- un pays ou une zone dans la liste commune à ONU M49 et aux groupes HDX

canoniques de toutes ces familles ;

- une politique de qualité COD-AB : amélioré uniquement, ou amélioré avec standard officiel en repli ;

- le format géospatial COD-AB : GeoJSON, GeoPackage, Shapefile ou File Geodatabase ;
- un intervalle d'actualisation entre 60 minutes et 30 jours ;
- la synchronisation automatique, ou **Synchroniser maintenant**.

Chaque passage interroge cod-ab-* et/ou cod-ps-*, exige l'identifiant exact <famille>-<iso3> ou la série officielle correspondante, puis vérifie l'unique groupe ISO3 contre le pays M49 choisi. COD-AB utilise le format demandé ; COD-PS retient ses ressources CSV/XLSX. Les codes M49, ISO3, famille, niveau COD publié, éditeur et licence restent associés aux ressources locales.

Le profil est atomique : si le pays n'est plus admissible dans une famille, HDP archive les réponses CKAN et le motif, mais ne télécharge pas uniquement les autres familles. Choisissez une nouvelle option ou retirez explicitement la famille devenue indisponible.

Après modification des familles, du pays, de la politique ou du format, le dernier état devient « synchronisation requise ». L'ancienne erreur disparaît ; utilisez ensuite **Synchroniser maintenant** ou laissez le planificateur exécuter le profil si le téléchargement automatique est actif.

Données locales

La rubrique **Données locales** présente les compteurs, la taille totale et chaque ressource.

- **Télécharger** remet le fichier local au navigateur.
- **Vérifier SHA-256** recalcule l'empreinte sans charger tout le fichier en mémoire.
- **Supprimer localement** demande confirmation, efface le fichier et marque la ressource deleted. L'acquisition et la ligne de provenance restent en base.

Scripts

Un script possède un nom, un langage, une description et un contenu. Chaque création ou modification produit une version immuable avec empreinte SHA-256. Python est activé par défaut ; R doit être activé dans les paramètres du projet et démarré avec `start-hdp-with-r.cmd`. SQL, shell et « autre » restent stockés mais ne sont pas exécutables.

Le bouton **Exécuter** crée un job asynchrone et affiche son état, sa sortie et son rapport JSON. Le délai est limité à 300 secondes et la sortie à 1 Mio. Le réseau des runners est toujours désactivé dans cette itération : toute demande d'activation ou d'allowlist est refusée. Exécutez uniquement du code local de confiance ; les limites techniques ne constituent pas une sandbox multi-tenant.

Veille RSS

L'onglet **Veille RSS** propose les flux officiels ReliefWeb « mises à jour », « catastrophes », « emplois » et « formations ». Un abonnement appartient au projet actif, peut contenir une recherche et une langue, puis être lu immédiatement ou toutes les 15 minutes à 30 jours. HDP borne la réponse à 2 Mio, refuse les redirections hors du domaine autorisé et déduplique les éléments.

Chronologie

L'onglet **Chronologie** affiche sous forme de Gantt les acquisitions, passages de planification, exécutions Python/R et prochains passages du projet. Le sélecteur permet de couvrir de 1 à 365 jours.

Carte PostGIS

Une ressource locale GeoJSON complète peut être importée depuis **Données locales**. L'onglet **Carte** affiche ensuite la couche PostGIS avec Leaflet et permet de télécharger une archive contenant le GeoJSON, un script QGIS et un script R. Leaflet est livré localement. Le fond de carte OpenStreetMap est désactivé par défaut et ne contacte le serveur de tuiles qu'après clic sur **Activer le fond OSM**.

Planifications

Une planification enregistre une source, une requête, une limite de résultats, un intervalle et l'option de téléchargement. L'intervalle minimal est de 15 minutes.

- **Exécuter maintenant** attend l'acquisition et affiche son résultat.
- **Suspendre/Réactiver** modifie l'état sans perdre la définition.
- **Archiver** désactive la planification et conserve son historique.

ReliefWeb nécessite toujours un appname valide. Respectez les quotas : un intervalle minimal techniquement accepté peut être inadapté à une requête large, notamment lorsqu'un connecteur filtre localement un catalogue mondial.

Installation

Installation et mise à niveau Windows

Prérequis

- Windows 10 ou 11 x64 ;
- virtualisation matérielle et WSL 2 disponibles ;
- Docker Desktop opérationnel ;
- au moins 10 Gio libres recommandés ;
- accès Internet aux registres Docker et aux sources de données.

L'installateur peut proposer Docker Desktop, Git et Visual Studio Code via winget. Aucune case n'est cochée automatiquement. Le module R est facultatif.

Installation

- Décompressez HumanitarianDataPlatform_Windows_v3.0.0.zip.
- Vérifiez l'empreinte de l'exécutable :

```
``powershell Get-FileHash .\HumanitarianDataPlatform_Setup_Native_GUI_v3.0.0.exe -Algorithm
SHA256 ``
```

- Comparez le résultat au contenu de HumanitarianDataPlatform_Setup_Native_GUI_v3.0.0.exe.sha256.
- Lancez l'exécutable et vérifiez le dossier proposé :

```
``text %USERPROFILE%\HumanitarianDataPlatform ``
```

- Saisissez un apnnome ReliefWeb uniquement s'il a été pré-approuvé.
- Facultatif : saisissez un jeton GitHub autorisé à créer les dépôts souhaités. Le champ est masqué.
- Sélectionnez explicitement les composants voulus, puis installez.

L'installateur choisit 8080 si possible, sinon un port libre entre 18080 et 18279. La valeur est enregistrée dans .env et utilisée par Compose, les scripts et le navigateur.

Mise à niveau depuis 2.5.0

Relancez l'installateur 3.0.0 en conservant le même dossier. Il remplace les fichiers applicatifs embarqués mais préserve :

- .env, donc le mot de passe PostgreSQL, l'apnnome ReliefWeb, le jeton GitHub,

le port et toute autre variable inconnue de l'installateur ;

- le volume nommé postgres_data ;
- le dossier data, y compris les réponses brutes ;
- les images Docker déjà téléchargées.

Au premier démarrage, l'API effectue la migration idempotente et enregistre sa version. Voir [Migration vers 3.0.0](#). La compatibilité de la structure 2.5.0 est couverte par les contrats de l'itération 1. La qualification des versions 1.5 à 2.4 nécessite les archives ou sauvegardes correspondantes.

Après une mise à niveau depuis 2.3.x, vérifiez le profil géographique de chaque projet. Un profil déjà limité à un pays ou une zone est conservé avec COD-AB sélectionné. Une portée monde ou région est suspendue : choisissez explicitement un pays ou une zone dans la nouvelle liste ONU M49 × HDX, puis enregistrez.

L'activation de COD-PS peut réduire la liste : au 7 août 2026, Algérie est disponible pour COD-AB mais pas pour COD-PS, tandis que Soudan est disponible pour les deux. HDP ne remplace jamais automatiquement un territoire devenu incompatible.

Démarrage et arrêt

- `start-hdp.cmd` : cœur FastAPI/PostgreSQL et runner Python sans réseau ;
- `start-hdp-with-r.cmd` : ajoute R/plumber et le runner R sans réseau ;
- `stop-hdp.cmd` : arrête les conteneurs sans supprimer les volumes ni les fichiers.

La passerelle GitHub locale est démarrée avec le cœur sur 127.0.0.1:8091 par défaut. Elle reste en lecture seule fonctionnelle tant que `GITHUB_API_WRITE_ENABLED=false`.

Répertoires

Élément	Emplacement
Application	%USERPROFILE%\HumanitarianDataPlatform
Configuration	%USERPROFILE%\HumanitarianDataPlatform\.env
JSON bruts	%USERPROFILE%\HumanitarianDataPlatform\data\raw
Ressources	%USERPROFILE%\HumanitarianDataPlatform\data\projects
Rapports d'exécution	%USERPROFILE%\HumanitarianDataPlatform\data\projects\<projet>\executions
Journaux installateur	%LOCALAPPDATA%\HumanitarianDataPlatform\logs

Ne supprimez pas le volume PostgreSQL ou data/ sans sauvegarde explicite.

Architecture

Architecture, données et API

Vue d'ensemble

```
flowchart TD
    U["Interface web locale"] --> A["FastAPI 3.0.0"]
    A --> P["PostgreSQL + PostGIS"]
    A --> F["Fichiers et rapports"]
    A --> X["Sources distantes"]
    U --> G["Passerelle GitHub locale"]
    A --> Q["Spool d'exécution"]
    Q --> Y["Runner Python sans réseau"]
    Q -- facultatif --> R["Runner R sans réseau"]
```

Docker Compose orchestre l'API, PostgreSQL/PostGIS, le runner Python et les services R facultatifs. Les ports HTTP de l'API et de la passerelle GitHub sont publiés uniquement sur 127.0.0.1. PostgreSQL et R/plumber restent dans le réseau interne Compose ; les deux runners utilisent `network_mode: none`.

Modèle par projets

Entité	Rôle	Suppression
projects	Conteneur fonctionnel	Archivage logique ; le projet par défaut est protégé
project_preferences	Limites et téléchargement automatique par défaut	Suit le projet
project_github_settings	Propriétaire, dépôt, description, visibilité et URL créée ; aucun jeton	Suit le projet
project_geodata_settings	Familles COD, pays/zone M49, politique, format et cycle	Automatisation désactivée à l'archivage
source_global_settings	Activation, délai et reprises par connecteur	Conservé globalement
project_source_settings	Contrat API, valeurs et planification par projet/source	Suit le projet
schema_migrations	Historique des migrations appliquées	Conservé
acquisitions	Provenance de chaque réponse distante	Conservée
local_resources	URL, empreinte et provenance famille/M49/ISO3/COD/licence	Fichier supprimé, ligne marquée deleted
project_scripts	État courant des scripts par projet	Archivage logique
project_execution_settings	Activation Python/R et limites du projet	Suit le projet
script_versions	Versions immuables et empreintes du code	Conservées avec le script
script_executions	État, sorties et rapport SHA-256	Historique conservé
rss_subscriptions / rss_items	Veille officielle et éléments dédupliques	Abonnement archivable
map_layers / map_features	Couches et géométries PostGIS SRID 4326	Suit le projet
schedules	Définition périodique d'une acquisition	Désactivation et archivage logique
schedule_runs	Historique des passages	Conservé avec statut et erreur éventuelle

Le projet par défaut utilise l'UUID stable 00000000-0000-4000-8000-000000000001. Le démarrage crée les nouvelles tables de façon idempotente, ajoute `project_id` et `schedule_id` à l'historique v1.5, puis rattache les lignes sans projet.

Intégrations de projet 3.0.0

Le registre `source_registry.py` décrit chaque API au moyen d'un contrat versionné inspiré de JSON Schema. L'interface génère les champs depuis ce contrat. L'API valide à nouveau toutes les valeurs, fusionne le modèle du projet avec les valeurs d'exécution, applique les limites globales et archive la configuration effective avec la réponse brute.

La création GitHub utilise `POST /user/repos` lorsque le propriétaire est vide ou correspond au compte du jeton, sinon `POST /orgs/{org}/repos`. Le dépôt est privé par défaut et initialisé avec un README. `GITHUB_TOKEN` demeure une variable d'environnement globale ; il ne transite jamais dans les modèles de réponse.

Le profil appelle `package_search` pour les catalogues canoniques `cod-ab-*` et `cod-ps-*`. Chaque jeu doit correspondre à la série officielle ou à l'identifiant exact `<famille>-<iso3>`, avec un unique groupe ISO3 connu de ONU M49. COD-AB exige `cod-enhanced` ou `cod-standard`; COD-PS accepte l'absence de ce champ, qui n'est pas publié uniformément pour cette série.

`GET /api/cod/availability` calcule l'intersection des ISO3 valides pour les familles sélectionnées, la convertit en pays ou zones M49 et met les catalogues en cache pendant 30 minutes. Au 7 août 2026, la même règle retourne 163 pays ou zones COD-AB, 146 COD-PS et 143 dans leur intersection. COD-CS utilise un registre versionné, actuellement vide et donc non sélectionnable. COD-HP est explicitement retiré et ne peut pas être ajouté au profil.

Lorsqu'une famille, un pays, une politique COD ou un format change, le dernier résultat est invalidé (`sync_required`) avant toute nouvelle synchronisation. L'interface ne peut ainsi pas associer l'erreur d'un ancien pays au nouveau périmètre.

Les réponses CKAN complètes, le pays M49, les familles, les jeux retenus, les absences et les formats manquants sont archivés sous `hdx-geodata`. Si une famille sélectionnée manque, aucun sous-ensemble n'est téléchargé. Une ressource déjà complète ne consomme pas le quota ; les suivantes obtiennent le compte `deferred` et sont reprises lors d'un passage ultérieur.

Flux d'acquisition et de téléchargement

- L'API valide le projet, la source, la requête et la limite.
- Elle appelle le connecteur choisi : ReliefWeb, HDX/CKAN, OMS/GHO,

Banque mondiale/WDI, UNICEF/SDMX, ONU/ODD ou DHS.

- La réponse brute est sérialisée en UTF-8, archivée et hachée en SHA-256.
- Une ligne acquisitions conserve la provenance.
- Si le téléchargement est actif, l'API extrait les ressources référencées.
- Elle applique les préférences : nombre maximal, taille maximale et formats autorisés.
- Chaque URL et chaque redirection doivent viser une adresse IP publique.
- Le fichier est écrit sous forme `.part`, contrôlé pendant le flux, puis renommé atomiquement.
- Le chemin relatif, la taille, le type de contenu et l'empreinte SHA-256 sont enregistrés

```
data/raw/<project_uuid>/<source>/<horodatage>_<requete>_<acquisition_uuid>.json
data/projects/<project_uuid>/resources/<acquisition_uuid>/<resource_uuid>_<nom>
```

Exécution locale des scripts

L'API écrit atomiquement un job dans un volume `spool`. Un runner C distinct le réclame par renommage, lance directement Python ou R avec `execve` — jamais via un shell — puis écrit statut, sortie et horodatages. Le collecteur persiste le résultat et un rapport JSON sous `data/projects/<projet>/executions`.

Les conteneurs sont non privilégiés, en lecture seule hors `/tmp` et du `spool`, sans réseau, avec limites de processus, mémoire, CPU, durée, descripteurs et taille de sortie. Cette frontière vise une application locale mono-utilisateur : elle ne transforme pas HDP en plateforme d'exécution de code hostile.

RSS, cartographie et chronologie

Le registre RSS ne contient que quatre URL ReliefWeb vérifiées. Les lectures sont bornées, suivent uniquement les hôtes autorisés, utilisent ETag et Last-Modified, refusent DTD/entités XML et dédupliquent par identifiant externe.

L'import cartographique accepte un Feature ou FeatureCollection GeoJSON borné, stocke les géométries en PostGIS et expose un FeatureCollection limité à 5 000 entités. Leaflet 1.9.4 est servi depuis le payload local. Le fond OSM reste opt-in. L'export produit GeoJSON et scripts d'import QGIS/R.

Passerelle GitHub

Le service github-api expose sur le port local 8091 un sous-ensemble explicite de l'API REST GitHub. Les lectures couvrent dépôt, branches, commits, issues, pull requests, releases, workflows, contenus et quota. Les deux écritures admises - création d'issue et déclenchement manuel de workflow - passent par un verrou serveur désactivé par défaut. Le jeton n'est jamais renvoyé au client.

Cette passerelle est séparée de la création confirmée d'un dépôt dans l'API principale. Son conteneur est non privilégié, en lecture seule et lié à 127.0.0.1. Elle ne doit pas être exposée sur un réseau partagé.

Planificateur

Le planificateur est une tâche de fond de l'unique processus Uvicorn. Il interroge PostgreSQL toutes les 20 secondes, collecte les jobs Python/R et revendique les acquisitions, synchronisations géographiques ou lectures RSS dues avec FOR UPDATE SKIP LOCKED. Le prochain passage est avancé avant l'appel distant et le résultat est persisté.

L'intervalle est compris entre 15 minutes et 30 jours. Une erreur de base temporaire ne termine pas définitivement la boucle. L'architecture suppose un seul processus API ; déployer plusieurs workers nécessiterait un service de tâches dédié.

API locale principale

Méthode et route	Fonction
GET /api/health	Santé SQL, version et état du planificateur
GET /api/sources	Catalogue des connecteurs actifs et portails de référence
GET/POST /api/projects	Liste et création des projets
PATCH/DELETE /api/projects/{id}	Modification ou archivage logique
GET/PUT /api/projects/{id}/preferences	Préférences de téléchargement
GET/PUT /api/projects/{id}/github	Paramètres GitHub sans secret
POST /api/projects/{id}/github/repository	Création confirmée du dépôt
GET/PUT /api/projects/{id}/geodata	Profil géographique et état de synchronisation
POST /api/projects/{id}/geodata/sync	Synchronisation HDX immédiate
GET /api/cod/families	Liste, état sélectionnable/retiré et registre COD-CS
GET /api/cod/availability	Intersection pays/zone ONU M49 x familles HDX
GET /api/un-m49/entities	Nomenclature hiérarchique M49 et source d'autorité
GET /api/search	Acquisition manuelle et téléchargement optionnel
GET /api/acquisitions?project_id=...	Historique du projet
GET /api/resources?project_id=...	Inventaire local
GET /api/resources/{id}/file	Téléchargement depuis le stockage local
POST /api/resources/{id}/verify	Recalcul SHA-256 en flux
DELETE /api/resources/{id}	Suppression locale avec trace conservée
GET/POST /api/projects/{id}/scripts	Bibliothèque de scripts

Méthode et route	Fonction
PATCH/DELETE /api/scripts/{id}	Modification ou archivage d'un script
GET /api/scripts/{id}/versions	Versions immuables
GET/POST /api/scripts/{id}/executions	Historique et lancement Python/R
GET /api/executions/{id}	État et sortie d'un job
GET /api/executions/{id}/report	Rapport JSON de l'exécution
GET/PUT /api/projects/{id}/execution-settings	Limites et activation des runners
GET /api/rss/catalog	Registre RSS officiel
GET/POST /api/projects/{id}/rss/subscriptions	Abonnements du projet
POST /api/rss/subscriptions/{id}/fetch	Lecture RSS immédiate
POST /api/resources/{id}/map/import	Import GeoJSON dans PostGIS
GET /api/projects/{id}/map/layers	Couches cartographiques
GET /api/map/layers/{id}/geojson	Couche GeoJSON bornée
GET /api/map/layers/{id}/export	Archive QGIS/R
GET /api/projects/{id}/timeline	Événements Gantt
GET/POST /api/projects/{id}/schedules	Liste et création des planifications
PATCH/DELETE /api/schedules/{id}	Modification ou archivage
POST /api/schedules/{id}/run	Exécution manuelle immédiate
GET /api/schedules/{id}/runs	Historique d'exécution
GET /docs	Swagger UI générée par FastAPI

GET /api/search et POST /api/schedules/{id}/run ont des effets : ils créent une acquisition et peuvent télécharger des fichiers.

Service R

Le profil analytics fournit R/plumber avec /health et /summary, ainsi que le runner R sans réseau. Les deux restent facultatifs.

Références des sources distantes

- [CKAN Action API](#) : actions package_search, réponses success/result et métadonnées de ressources ;
- [ONU M49](#) : nomenclature statistique du monde, des régions, pays et zones ;
- [OCHA COD-AB](#) : limites administratives communes ;
- [OCHA Common Operational Datasets](#) : familles COD actuelles et retrait de COD-HP ;
- [OCHA COD-CS](#) : nature contextuelle des données spécifiques au pays ;
- [GitHub REST — repositories](#) : création pour un utilisateur ou une organisation ;
- [ReliefWeb RSS](#) : flux officiels enregistrés ;
- [Leaflet 1.9.4](#) et [politique des tuiles OpenStreetMap](#) : rendu embarqué et fond opt-in ;
- [ReliefWeb API V2](#) et [paramètres](#) : appname, profils, requêtes, limites et quotas.
- [WHO GHO OData API](#),

[World Bank Indicators API](#), [UNICEF SDMX API](#), [UN SDG API](#) et [DHS Program API](#) : catalogues d'indicateurs et flux sanitaires ajoutés en 2.5.0.

Référence API

Référence API - Humanitarian Data Platform 3.0.0

API principale

Base locale : `http://127.0.0.1:<HDP_PORT>`, 8080 par défaut. Swagger : `/docs`. OpenAPI : `/openapi.json`.

Système et sources

- GET `/api/health`
- GET `/api/sources`
- GET `/api/source-settings`
- GET/PUT `/api/source-settings/{source_id}`
- GET `/api/projects/{project_id}/sources`
- GET/PUT `/api/projects/{project_id}/sources/{source_id}`
- POST `/api/projects/{project_id}/sources/{source_id}/preview`
- GET `/api/search`

Projets et intégrations

- GET/POST `/api/projects`
- PATCH/DELETE `/api/projects/{project_id}`
- GET/PUT `/api/projects/{project_id}/preferences`
- GET/PUT `/api/projects/{project_id}/github`
- POST `/api/projects/{project_id}/github/repository`
- GET/PUT `/api/projects/{project_id}/geodata`
- POST `/api/projects/{project_id}/geodata/sync`
- GET `/api/cod/families`
- GET `/api/cod/availability`
- GET `/api/un-m49/entities`

Acquisitions et ressources

- GET `/api/acquisitions?project_id=...`
- GET `/api/resources?project_id=...`
- GET `/api/resources/{resource_id}/file`
- POST `/api/resources/{resource_id}/verify`
- DELETE `/api/resources/{resource_id}`
- GET `/api/projects/{project_id}/storage`

Scripts et exécutions

- GET/POST `/api/projects/{project_id}/scripts`
- PATCH/DELETE `/api/scripts/{script_id}`
- GET `/api/scripts/{script_id}/versions`

- GET/POST /api/scripts/{script_id}/executions
- GET /api/executions/{execution_id}
- GET /api/executions/{execution_id}/report
- GET/PUT /api/projects/{project_id}/execution-settings

Planifications, RSS et chronologie

- GET/POST /api/projects/{project_id}/schedules
- PATCH/DELETE /api/schedules/{schedule_id}
- POST /api/schedules/{schedule_id}/run
- GET /api/schedules/{schedule_id}/runs
- GET /api/rss/catalog
- GET/POST /api/projects/{project_id}/rss/subscriptions
- PATCH/DELETE /api/rss/subscriptions/{subscription_id}
- POST /api/rss/subscriptions/{subscription_id}/fetch
- GET /api/projects/{project_id}/rss/items
- GET /api/projects/{project_id}/timeline

Cartographie

- GET /api/map/config
- POST /api/resources/{resource_id}/map/import
- GET /api/projects/{project_id}/map/layers
- GET /api/map/layers/{layer_id}/geojson
- GET /api/map/layers/{layer_id}/export

GET /api/search, les synchronisations et les exécutions manuelles ont des effets persistants. Les réponses d'erreur FastAPI utilisent detail.

Passerelle GitHub

Base locale : `http://127.0.0.1:<HDP_GITHUB_API_PORT>, 8091` par défaut. Swagger : `/docs`. OpenAPI : `/openapi.json`.

- GET /health
- GET /repository
- GET /branches
- GET /commits
- GET /issues
- POST /issues, verrouillé par défaut
- GET /pulls
- GET /releases
- GET /workflows
- POST /workflows/{workflow_id}/dispatch, verrouillé par défaut
- GET /contents/{content_path}
- GET /rate-limit

Les routes acceptent owner et repo facultatifs. Les listes utilisent page et per_page borné à 100. Voir [GITHUB_API.md](#) pour les permissions et la configuration.

Passerelle GitHub

Passerelle REST GitHub - HDP 3.0.0

HDP embarque le service Docker `github-api`, une passerelle locale et bornée vers les fonctions REST GitHub utilisées par le projet. Elle complète la création de dépôt disponible dans l'API principale.

Sécurité

- le service est publié uniquement sur `127.0.0.1` ;
- `GITHUB_TOKEN` reste côté serveur et n'est jamais renvoyé ;
- les lectures sont disponibles par défaut ;
- les écritures sont désactivées par défaut ;
- le conteneur est non privilégié, en lecture seule, avec limites de mémoire, CPU et processus ;

- propriétaire, dépôt, workflow, pagination et tailles de corps sont validés ;
- l'API REST est appelée avec `X-GitHub-API-Version: 2026-03-10` par défaut.

HDP demeure une application locale sans authentification. Ne publiez jamais le port 8091 sur le réseau local ou Internet.

Configuration

Variables optionnelles dans `.env` :

```
GITHUB_TOKEN=
GITHUB_API_VERSION=2026-03-10
GITHUB_DEFAULT_OWNER=B-DAUTRIF
GITHUB_DEFAULT_REPOSITORY=humanitarian-data-platform
GITHUB_API_WRITE_ENABLED=false
GITHUB_API_TIMEOUT_SECONDS=20
HDP_GITHUB_API_PORT=8091
```

Ne versionnez jamais un jeton réel. Préférez un jeton finement granulé limité au propriétaire et aux dépôts nécessaires.

Endpoints locaux

Méthode	Route	Fonction
GET	<code>/health</code>	Version, état du jeton et verrou d'écriture
GET	<code>/repository</code>	Métadonnées du dépôt
GET	<code>/branches</code>	Branches paginées
GET	<code>/commits</code>	Commits paginés, filtre sha facultatif
GET	<code>/issues</code>	Issues paginées
GET	<code>/pulls</code>	Pull requests paginées
GET	<code>/releases</code>	Releases paginées
GET	<code>/workflows</code>	Workflows GitHub Actions
GET	<code>/contents/{path}</code>	Contenu ou métadonnées d'un chemin
GET	<code>/rate-limit</code>	Quotas du jeton ou de l'adresse appelante
POST	<code>/issues</code>	Création d'une issue, verrouillée par défaut

Méthode	Route	Fonction
POST	/workflows/{id}/dispatch	Déclenchement manuel, verrouillé par défaut

Les listes acceptent page et per_page entre 1 et 100. Les métadonnées de pagination et de quota sont retournées sans recopier d'en-tête sensible.

Activation des écritures

Les écritures nécessitent simultanément :

- GITHUB_API_WRITE_ENABLED=true ;
- un GITHUB_TOKEN configuré ;
- les permissions GitHub minimales adaptées à l'opération.

Pour un jeton finement granulé :

- création d'issue : permission **Issues: write** sur le dépôt visé ;
- déclenchement de workflow : permission **Actions: write** ;
- création d'un dépôt depuis l'API principale : permission

Administration: write.

N'accordez pas **Contents: write** tant qu'aucune publication de fichiers par l'application n'est prévue.

Exemples locaux

```
curl http://127.0.0.1:8091/health
curl http://127.0.0.1:8091/repository
curl "http://127.0.0.1:8091/branches?per_page=20&page=1"
curl "http://127.0.0.1:8091/commits?sha=main"
curl "http://127.0.0.1:8091/issues?state=open"
curl http://127.0.0.1:8091/workflows
```

Références : [API REST GitHub](#), [versionnement](#) et [jetons personnels](#).

Migration depuis 2.5.0

Migration de 2.5.0 vers 3.0.0

Garanties

La migration est déclenchée au démarrage et peut être rejouée. Elle ne supprime ni fichiers, ni volumes, ni lignes d'acquisition.

- création des tables `projects`, `project_preferences`, `project_scripts`, `schedules`, `schedule_runs` et `local_resources` si elles sont absentes ;
- ajout de `project_id` et `schedule_id` à `acquisitions` ;
- création du « Projet par défaut » ;
- rattachement des acquisitions sans projet à ce projet ;
- activation de la contrainte `NOT NULL` sur `acquisitions.project_id`.
- création idempotente de `project_github_settings` et `project_geodata_settings`, puis ajout d'une ligne par projet existant ;
- ajout idempotent de `m49_scope_code`, `official_policy` et `migration_required` ;
- ajout de `cod_families`, initialisé à `["cod-ab"]` pour les profils existants ;
- conservation des profils déjà limités à un pays ou une zone M49 ;
- suspension des profils monde/région et du téléchargement automatique jusqu'au choix explicite d'une option ONU M49 × HDX ;
- ajout de `cod_family` à la provenance des ressources locales, sans réécriture des anciennes lignes.
- création de `schema_migrations`, `source_global_settings` et `project_source_settings` ;
- ajout de `parameters` aux acquisitions et planifications ;
- ajout des champs de bibliothèque `subject`, `published_at`, `geographic_scope`, `resource_type`, `organization` et `metadata` ;
- initialisation des réglages globaux pour les 18 sources et des réglages de projet pour les 7 connecteurs, sans écraser une valeur déjà présente.
- création de `project_execution_settings`, `script_versions` et `script_executions`, puis création d'une version 1 pour chaque script existant ;
- création de `rss_subscriptions` et `rss_items` ;
- création de `map_layers` et `map_features`, avec géométrie PostGIS SRID 4326 et index GIST.

Une ancienne ligne de ressource garde `cod_family = NULL` si elle précède 2.4.0 ; son niveau COD et ses autres métadonnées restent intacts. Toute nouvelle ressource officielle reçoit `cod-ab` ou `cod-ps`.

Les anciens fichiers restent à leur emplacement `data/raw/<source>`. Leur colonne `raw_path` n'est pas réécrite. Les nouvelles acquisitions utilisent `data/raw/<project_uuid>/<source>`.

Les anciens scripts restent inchangés. Leur première version immuable est créée au démarrage ; seuls Python et R deviennent exécutables, avec réseau désactivé.

Sauvegarde recommandée

Avant une mise à niveau importante, arrêtez HDP et sauvegardez :

```
%USERPROFILE%\HumanitarianDataPlatform\.env
%USERPROFILE%\HumanitarianDataPlatform\data
volume Docker humanitarian-data-platform_postgres_data
```

Ne partagez pas .env. Pour sauvegarder le volume PostgreSQL, utilisez une procédure Docker adaptée à votre environnement ; n'employez pas **Clean/Purge data** ni **Reset to factory defaults**.

Retour à une version antérieure

Les anciennes versions ignorent les nouvelles tables, mais ne connaissent pas les nouvelles colonnes de provenance. Un retour applicatif sans restauration de sauvegarde n'est pas un scénario officiellement validé. Conservez une sauvegarde cohérente si un retour arrière est nécessaire.

Sécurité

Sécurité et confidentialité

Périmètre

HDP 3.0.0 reste une application locale mono-utilisateur. Elle n'a ni authentification, ni TLS local, ni séparation des rôles. Ne publiez pas son port sur le réseau et ne l'exposez pas à Internet.

Protections présentes

- liaison de l'API à 127.0.0.1 uniquement ;
- aucun port PostgreSQL publié sur Windows ;
- service R interne à Compose ; runners Python/R avec `network_mode: none` ;
- passerelle GitHub liée à 127.0.0.1, conteneur non privilégié et en lecture seule, écritures désactivées par défaut ;
- mot de passe PostgreSQL aléatoire conservé dans `.env` ;
- empreintes SHA-256 des JSON et ressources ;
- noms de fichiers neutralisés et chemins confinés sous `data/` ;
- téléchargements HTTP(S) seulement, sans identifiants intégrés ;
- résolution DNS et refus des adresses privées, locales, réservées ou non globales à chaque URL et redirection ;
- limite de taille contrôlée avec `Content-Length` puis pendant le flux ;
- écriture temporaire `.part` et renommage après réussite ;
- exécution limitée à Python/R, versions immuables, appel direct sans shell, conteneurs non privilégiés et en lecture seule, limites de temps, processus, CPU, mémoire, fichiers et sortie ;
- toute activation réseau ou allowlist d'un job est refusée ;
- parseur RSS borné et durci contre DTD/entités, redirections limitées aux hôtes du registre ;
- Leaflet servi localement ; fond OSM désactivé jusqu'à une action explicite ;
- jeton GitHub lu depuis l'environnement, jamais retourné par l'API ni enregistré dans les paramètres de projet ;
- passerelle GitHub limitée à des routes codées en dur : elle n'accepte ni URL arbitraire, ni méthode libre, ni retransmission de l'en-tête `Authorization` ;
- dépôt GitHub privé par défaut et création précédée d'une confirmation côté interface ;
- aucune saisie libre d'identifiant HDX dans le module géographique ; filtres stricts sur `cod-ab-<iso3>/cod-ps-<iso3>`, les séries officielles et l'unique groupe ISO3 M49 ;
- COD-CS désactivé lorsque le registre vérifié est vide et COD-HP impossible à sélectionner ;
- absence d'une famille traitée atomiquement, sans téléchargement silencieux d'un sous-ensemble ;
- suppressions de ressources confirmées dans l'interface et provenance conservée.

Limites connues

- absence d'audit de sécurité indépendant ;
- HTTP local et données en clair sur le disque ;

- pas de chiffrement applicatif ni rotation automatique des secrets ;
- pas d'antivirus ou d'analyse du contenu téléchargé ;
- le filtrage réseau réduit le risque SSRF mais ne remplace pas une isolation réseau ;
- dépendance aux métadonnées, licences, quotas et disponibilités des sources ;
- les connecteurs filtrant un catalogue distant peuvent transférer une réponse

volumineuse même lorsque peu de résultats sont affichés ;

- un marquage COD officiel réduit le périmètre mais ne remplace pas l'évaluation de la qualité ou de l'adéquation d'un jeu ;
- installateur non signé par certificat d'éditeur.
- les runners partagent un spool par langage et ne sont pas une sandbox

multi-tenant : n'exécutez que des scripts locaux de confiance ;

- activer le fond OpenStreetMap communique les requêtes de tuiles au service

distant et impose le respect de sa politique d'utilisation ;

Fichiers sensibles

%USERPROFILE%\HumanitarianDataPlatform\.env contient le secret PostgreSQL, éventuellement l'appname ReliefWeb et GITHUB_TOKEN. Ne publiez jamais ce fichier. Préférez un jeton finement granulé limité aux dépôts et organisations requis. La création de dépôt exige la permission de dépôt **Administration: write** ; n'accordez pas de droits de contenu supplémentaires tant qu'aucune publication n'est prévue et révoquez le jeton lorsqu'il n'est plus nécessaire.

Le diagnostic HDP_Diagnostic_v3.0.0.cmd n'affiche que HDP_PORT depuis .env : ni le jeton GitHub, ni le mot de passe PostgreSQL, ni l'appname ReliefWeb. Relisez néanmoins tout journal avant de le partager : il peut contenir le nom de la machine, l'utilisateur, des chemins ou des informations Docker.

Données humanitaires

N'utilisez pas HDP pour des données personnelles ou confidentielles sans évaluation adaptée. Les mots-clés sont transmis à la source choisie lorsqu'elle permet un filtrage distant ; les autres connecteurs téléchargent un catalogue puis filtrent localement. ReliefWeb peut associer les appels à l'appname.

Empreinte et signature

SHA-256 détecte une modification par rapport à une valeur attendue ; il ne prouve ni l'identité de l'éditeur, ni l'exactitude des données. Vérifiez les fichiers .sha256 obtenus via un canal de confiance.

Licence

Aucune licence HDP explicite n'est incluse. Le dépôt doit rester privé jusqu'au choix d'une licence et à la vérification des droits de redistribution des composants.

Développement

Développement et validation

Arborescence

```
source/payload/api/app/ API FastAPI, planificateur et sécurité
source/payload/api/static/ interface web autonome
source/payload/r-service/ module R facultatif
source/payload/runner/ runners C Python/R sans shell ni réseau
source/payload/github-api/ passerelle REST GitHub locale et verrouillée
source/src/ installateur C/Win32 et payload généré
source/scripts/ génération du tableau C
source/tests/ tests Python et roundtrip C
```

Exécution en développement

Créez `.env` à côté de `compose.yaml` :

```
POSTGRES_PASSWORD=une-valeur-aleatoire
RELIEFWEB_APPNAME=
GITHUB_TOKEN=
HDP_PORT=8080
```

Puis :

```
cd source/payload
docker compose up -d --build db runner-python github-api api
```

Tests

```
python -m compileall -q source/payload/api/app
python -m unittest discover -s source/tests -p 'test_*.py' -v
gcc -std=c17 -O2 -Wall -Wextra -Werror source/payload/runner/runner.c -o /tmp/hdp-runner
node source/scripts/generate_payload.mjs source/payload source/src/payload_generated.h
```

Le test C de roundtrip doit être compilé après génération du payload. Il compare chaque chemin et chaque octet embarqué au contenu source. Les archives ZIP doivent passer `unzip -t` et toutes les valeurs de `SHA256SUMS.txt` doivent être recalculées après la création finale.

Construction Windows

Depuis `source/` :

```
bash build.sh /chemin/vers/zig /chemin/vers/cache
```

Le résultat est `HumanitarianDataPlatform_Setup_Native_GUI_v3.0.0.exe`. La compilation cible `x86_64-windows-gnu`, le sous-système GUI et traite les avertissements comme erreurs.

Validation non disponible dans un environnement sans Docker

La compilation Python, les tests unitaires et de contrat, la syntaxe JavaScript, le roundtrip du payload, le format PE, les archives et les empreintes peuvent être validés sans Docker. La migration PostgreSQL/PostGIS et le parcours navigateur complet nécessitent un moteur Docker opérationnel et doivent être exécutés sur la machine Windows de recette. L'EXE est compilable depuis Linux avec Zig ; sa recette Windows complète demeure distincte de la validation de compilation croisée.

Dépannage

Dépannage

Diagnostic borné

- Double-cliquez sur `HDP_Diagnostic_v3.0.0.cmd`.
- Attendez la fin ; chaque commande externe est limitée à 15 secondes.
- Récupérez `HDP_Debug_v3.0.0_*.log` sur le Bureau.
- Relisez et masquez toute information personnelle avant partage.

Problèmes courants

Symptôme	Cause probable	Action
Docker ne répond pas	Premier démarrage, WSL ou conditions Docker en attente	Ouvrir Docker Desktop, terminer son assistant, puis relancer
Port 8080 refusé	Port occupé ou réservé	Lire <code>HDP_PORT</code> dans <code>.env</code> ; l'installateur choisit 18080–18279
ReliefWeb retourne 503	RELIEFWEB_APPNAME absent	Obtenir un appname pré-approuvé, l'ajouter à <code>.env</code> , redémarrer
Création GitHub indisponible	GITHUB_TOKEN absent ou conteneur API créé avant la modification de <code>.env</code>	Double-cliquer sur <code>HDP_Configurer_GitHub_v3.0.0.cmd</code> , saisir le jeton dans l'invite masquée et attendre la vérification
Source sanitaire retourne 502	API tierce indisponible, quota, changement de schéma ou délai dépassé	Ouvrir le lien Documentation dans « Sources sanitaires », réessayer plus tard et conserver l'erreur exacte
Portail sanitaire non proposé dans Recherche	entrée classée « Référence »	Utiliser le lien Portail ; HDP n'affirme pas une API publique stable pour cette source
Création GitHub refusée	dépôt existant ou droits compte/organisation insuffisants	Choisir un autre nom ou ajuster les permissions du jeton
migration_required	ancien profil monde/région non représentable dans la liste commune	Choisir un pays ou une zone ONU M49 × HDX, puis enregistrer
Liste géographique plus courte	plusieurs familles sélectionnées	La liste est leur intersection ; retirer explicitement une famille ou choisir une option commune
Algérie absente avec COD-PS	aucun cod-ps-dza canonique vérifié au 7 août 2026	Sélectionner COD-AB seul ou un autre pays commun ; ne pas forcer un jeu non officiel
no_official_dataset	une famille sélectionnée a disparu du catalogue pour ce pays	Actualiser la liste, changer de pays ou retirer explicitement la famille
Nouveau périmètre avec l'erreur de l'ancien pays	installation 2.3.1 conservant le dernier statut	Mettre à niveau vers 2.4.0, enregistrer le profil puis synchroniser ; le statut devient d'abord « synchronisation requise »
Soudan ou Algérie indiqué sans COD-AB	filtre 2.3.1 exigeant dataseries_name, absent du JSON CKAN actuel	Mettre à niveau vers 2.4.0 et relancer la synchronisation
no_matching_resource	format COD-AB absent ou aucune ressource CSV/XLSX COD-PS	Changer le format COD-AB ou vérifier la publication officielle ; ne pas substituer un jeu arbitraire
Ressources deferred	limite de quantité atteinte	Attendre le prochain passage ou augmenter prudemment la limite du projet
Ressource failed	URL, réseau, taille, redirection privée ou source distante	Lire l'erreur dans Données locales et ajuster les préférences
Ressource ignorée	Limite de quantité ou format non autorisé	Modifier les préférences du projet
Planification jamais exécutée	Suspendue ou planificateur arrêté	Vérifier le badge de santé, réactiver puis consulter les journaux API

Symptôme	Cause probable	Action
Runner Python indisponible	runner-python absent ou encore en construction	Exécuter <code>docker compose up -d --build runner-python api</code> , puis vérifier <code>/api/health</code>
Runner R indisponible	Profil analytique non démarré	Utiliser <code>start-hdp-with-r.cmd</code> ou <code>docker compose --profile analytics up -d --build runner-r r-service api</code>
Exécution refusée pour le réseau	Politique de sécurité 3.0.0	Retirer l'activation réseau et l'allowlist ; seuls les jobs hors ligne sont admis
Flux RSS en échec	Réseau, redirection, taille ou XML distant invalide	Conserver l'erreur affichée, vérifier le flux officiel et réessayer plus tard
Carte sans fond	Comportement par défaut	Cliquer sur Activer le fond OSM uniquement si l'accès au service de tuiles est souhaité
Passerelle GitHub indisponible	Image absente ou port 8091 occupé	Vérifier <code>HDP_GITHUB_API_PORT</code> , puis exécuter <code>docker compose up -d --build github-api api</code>
Écriture GitHub refusée par la passerelle	Verrou serveur ou permission insuffisante	Vérifier <code>GITHUB_API_WRITE_ENABLED=true</code> et les droits minimaux du jeton sans jamais l'afficher
Empreinte invalide	Fichier modifié après téléchargement	Ne pas utiliser le fichier ; relancer une acquisition
Fichier local absent	Déplacement ou suppression externe	La provenance reste en base ; relancer l'acquisition si nécessaire

Commandes utiles

Dans `%USERPROFILE%\HumanitarianDataPlatform` :

```
docker compose ps --all
docker compose logs --no-color --tail 200 api db runner-python github-api
docker compose up -d --build db runner-python github-api api
```

N'utilisez pas `docker compose down -v`, **Clean/Purge data** ou **Reset to factory defaults** si les données doivent être conservées.

Correctif automatisé du jeton GitHub

Le fichier `HDP_Configurer_GitHub_v3.0.0.cmd` applique la procédure complète sous Windows :

- il cible `%USERPROFILE%\HumanitarianDataPlatform` par défaut ;
- il demande le jeton dans une saisie masquée et met à jour uniquement la clé `GITHUB_TOKEN` de `.env` ;
- il exécute `docker compose up -d --no-deps --force-recreate api` sans supprimer les volumes ni recréer la base ;
- il vérifie dans le conteneur que la variable est non vide, sans l'afficher ;
- il rouvre HDP sur le port défini par `HDP_PORT`.

Ne collez jamais le jeton dans un journal, une issue GitHub ou une conversation. Si GitHub refuse ensuite la création, vérifiez les permissions du jeton et l'autorisation de créer un dépôt pour le compte ou l'organisation choisis.

Livrables

Livrables et empreintes — version finale 3.0.0

La remise finale est publiée sous `dist/v3.0.0/`. Le fichier à télécharger en priorité est `HumanitarianDataPlatform_Archive_complete_v3.0.0.zip`.

Fichier	Rôle
<code>HumanitarianDataPlatform_Archive_complete_v3.0.0.zip</code>	archive globale de tous les livrables finaux
<code>HumanitarianDataPlatform_Archive_complete_v3.0.0.zip.sha256</code>	empreinte dédiée de l'archive globale
<code>HumanitarianDataPlatform_Windows_v3.0.0.zip</code>	paquet utilisateur Windows
<code>HumanitarianDataPlatform_Source_v3.0.0.zip</code>	sources reproductibles et tests
<code>HumanitarianDataPlatform_Setup_Native_GUI_v3.0.0.exe</code>	installateur Windows natif PE32+ GUI x64
<code>HumanitarianDataPlatform_Setup_Native_GUI_v3.0.0.exe.sha256</code>	empreinte de l'installateur
<code>Documentation_Humanitarian_Data_Platform_v3.0.0.html</code>	documentation consolidée consultable
<code>Notice_detaillée_Humanitarian_Data_Platform_v3.0.0.pdf</code>	documentation consolidée imprimable
<code>HDP_Prompt_production_global_v3.0.0.txt</code>	prompt autonome de production et reconstruction
<code>MANIFESTE_HumanitarianDataPlatform_v3.0.0.txt</code>	constitution, versions, contrôles et limites
<code>SHA256SUMS.txt</code>	sommes des livrables intégrés à l'archive globale

L'archive globale contient en outre la documentation Markdown, le cahier des charges, la référence API, les rapports de validation, la matrice de compatibilité, le registre de décisions et les journaux LOG-Huma.

Validation

```
cd dist/v3.0.0
sha256sum -c SHA256SUMS.txt
sha256sum -c HumanitarianDataPlatform_Archive_complete_v3.0.0.zip.sha256
unzip -t HumanitarianDataPlatform_Source_v3.0.0.zip
unzip -t HumanitarianDataPlatform_Windows_v3.0.0.zip
unzip -t HumanitarianDataPlatform_Archive_complete_v3.0.0.zip
```

L'archive globale ne peut pas contenir sa propre empreinte. Son fichier `.sha256` est donc fourni à côté et vérifié séparément.

Historique

Les répertoires `dist/v2.5.0`, `dist/v2.4.0`, `dist/v2.3.2`, `dist/v2.3.1`, `dist/v2.0` et `dist/v1.5` restent des références historiques et ne remplacent pas la remise finale 3.0.0.

Historique

Journal des versions

3.0.0 — version finale — 15 août 2026

- Gel du contrat applicatif et suppression des libellés de préversion.
- Intégration complète de la passerelle REST GitHub apparue sur main en 2.4.1, renforcée par un conteneur non privilégié et des écritures désactivées par défaut.
- Documentation finale consolidée : cahier des charges, architecture, sécurité, référence API, guide HTML/PDF et prompt global de production.
- Reconstruction de l'installateur Windows, des archives source/utilisateur et de l'archive complète avec manifeste et empreintes SHA-256.

3.0.0 — itération 2 — 15 août 2026

- Exécution Python/R par versions immuables dans des runners distincts, non privilégiés, sans réseau et avec limites de temps, processus et sortie.
- Rapports JSON d'exécution avec empreintes SHA-256 et historique par script.
- Registre de quatre flux RSS officiels ReliefWeb, abonnements planifiés, déduplication, requêtes conditionnelles et parseur XML durci.
- Vue chronologique de type Gantt par projet.
- Import GeoJSON dans PostGIS, visualisation Leaflet 1.9.4 embarquée et exports QGIS/R ; fond OpenStreetMap activé uniquement à la demande.
- Politique GitHub actualisée pour les jetons finement granulés et l'API REST versionnée 2026-03-10.
- 47 chemins et 63 opérations dans le contrat OpenAPI généré.

3.0.0 — itération 1 — 14 août 2026

- Registre versionné des paramètres pour les sept connecteurs API.
- Paramètres globaux et modèles propres à chaque source et projet.
- Prévisualisation assainie des URL/commandes sans appel réseau.
- Paramètres effectifs archivés avec acquisitions et planifications.
- Métadonnées et filtres initiaux de bibliothèque.
- Migrations idempotentes enregistrées, conservation renforcée de .env.
- Chaîne Windows portée en 3.0.0 ; recette réelle reportée au jalon final selon D09=B.
- 52 tests Python réussis, plus syntaxe Python/JavaScript et roundtrip du payload.

2.5.0 — 7 août 2026

- Catalogue intégré de 18 sources sanitaires et épidémiologiques mondiales.

- Connecteurs interrogeables et planifiables ajoutés pour OMS/GHO, Banque mondiale/WDI, UNICEF/SDMX, ONU/ODD et DHS, en plus de HDX et ReliefWeb.
- Archivage SHA-256 de la réponse distante brute conservé pour tous les nouveaux connecteurs ; normalisation des indicateurs et flux vers le modèle HDP.
- Onglet « Sources sanitaires » distinguant 7 API actives de 11 portails de référence avec accès, domaines, inscription et liens officiels.
- 36 tests unitaires, compilation Python et syntaxe JavaScript validés.

2.4.1 — 14 août 2026

- Passerelle locale `github-api` vers l'API REST GitHub classique.
- Lectures de dépôt, branches, commits, issues, pull requests, releases, workflows, contenus et quotas.
- Création d'issues et déclenchement de workflows désactivés par défaut.
- Jeton conservé côté serveur et version REST configurable.

2.4.0 — 7 août 2026

- Présentation des téléchargements officiels sous forme de liste par famille : COD-AB et COD-PS sont sélectionnables dans un même profil de projet.
- COD-CS reste visible mais désactivé tant que son registre vérifié embarqué ne contient aucun jeu ; COD-HP est visible comme famille retirée par OCHA.
- Remplacement du périmètre hiérarchique libre par une liste de pays ou zones appartenant simultanément à ONU M49 et à tous les catalogues HDX sélectionnés.
- Vérification réelle du 7 août 2026 : 163 pays/zones COD-AB, 146 COD-PS et 143 options dans leur intersection ; Soudan admissible aux deux, Algérie au seul COD-AB.
- Téléchargement atomique par ensemble de familles : si une famille devient absente, la preuve CKAN est archivée mais aucun sous-ensemble n'est téléchargé.
- Ajout de `cod_families` aux profils et de `cod_family` à la provenance locale ; migration sûre des anciens profils pays, suspension explicite des profils monde/région jusqu'au choix dans la nouvelle liste.
- Ajout des routes `/api/cod/families` et `/api/cod/availability`, avec cache HDX de 30 minutes et registre COD-CS versionné.
- Validation : 29 tests unitaires, contrôle JavaScript et recette catalogue HDX en direct sur COD-AB/COD-PS.

2.3.2 — 7 août 2026

- Correction de la découverte HDX : le catalogue est interrogé par identifiant canonique `cod-ab-*` et niveau COD, puis chaque résultat est contrôlé contre son groupe ISO3 ONU M49.
- Compatibilité avec les réponses CKAN qui indexent la série officielle mais

n'exposent plus `dataseries_name` dans le JSON retourné.

- Invalidation du dernier statut, de la dernière erreur et de l'acquisition affichés lorsqu'un profil change de périmètre M49, de politique ou de format.
- Affichage explicite « synchronisation requise » et échéance immédiate lorsque le téléchargement automatique est actif sur un profil modifié.
- Régressions vérifiées sur les COD-AB officiels du Soudan (M49 729, SDN) et de l'Algérie (M49 012, DZA).

2.3.1 — 7 août 2026

- Remplacement de l'échelle opérationnelle « terrain → monde » par la nomenclature officielle ONU M49 embarquée et hiérarchisée.
- Suppression de l'identifiant HDX libre dans le module géographique.
- Limitation stricte aux jeux OCHA/HDX de la série officielle
COD - Subnational Administrative Boundaries, marqués `cod-enhanced` ou `cod-standard`.
- Ajout des politiques « amélioré uniquement » et « amélioré avec standard en repli ».
- Archivage de la provenance : code M49, ISO3, niveau COD, éditeur, licence et date des métadonnées.
- Migration sûre : les anciens profils monde deviennent M49 001; les profils plus étroits sont suspendus jusqu'au choix explicite d'un territoire M49.
- Correction de la limite d'acquisition : les fichiers déjà présents ne consomment plus le quota et les ressources restantes sont reportées, pas assimilées à des échecs.

2.3.0 — 7 août 2026

- Paramètres GitHub par projet et création de dépôt.
- Profil géographique HDX, synchronisation planifiée et téléchargement local.
- Correctif Windows de configuration sûre de `GITHUB_TOKEN`.

Les journaux détaillés historiques restent également inclus dans leurs archives de remise respectives sous `dist/`.